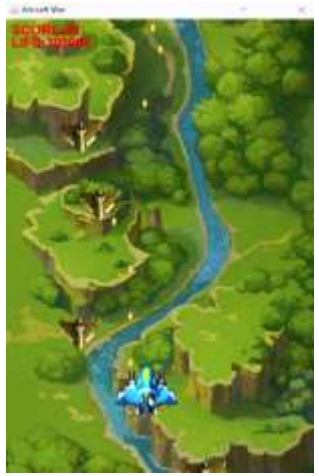
实验五：Swing和多线程

实验与创新实践教育中心 • 计算机与数据技术实验教学部

本学期实验总体安排

初始版本



最终版本

初始版本

游戏主界面
英雄机移动
英雄机子弹直射
碰撞检测
统计得分和生命值

01

UML建模
精英敌机生成、攻击
道具生成、加血道具生效
采用**单例模式**创建英雄机

02

采用**简单工厂模式**
创建道具
采用**工厂方法模式**
创建敌机

03

采用**策略模式**
实现不同飞机的发射弹道
两种火力道具生效

04

采用**数据访问对象模式**
实现得分排行榜
添加**JUnit单元测试**

05

使用**Swing**添加游戏难度选择和排行榜界面
使用**多线程**实现音效控制及火力道具状态恢复

06

采用**观察者模式**
实现炸弹、冰冻道具
采用**模板模式**
实现三种游戏难度

本学期实验总体安排

实验项目	一	二	三	四	五	六
学时数	2	2	2	2	4	4
实验内容	系统分析 单例模式	简单工厂 工厂方法模式	策略模式	DAO模式 单元测试	Swing 多线程	观察者模式 模板模式
时间节点			**中期**			**结项**
提交内容			UML类图 项目代码			结项报告 代码、类图 展示视频

实验课程共 16 个学时，6 个实验项目，总成绩为 30分 。

目录

CONTENTS

01 实验目的

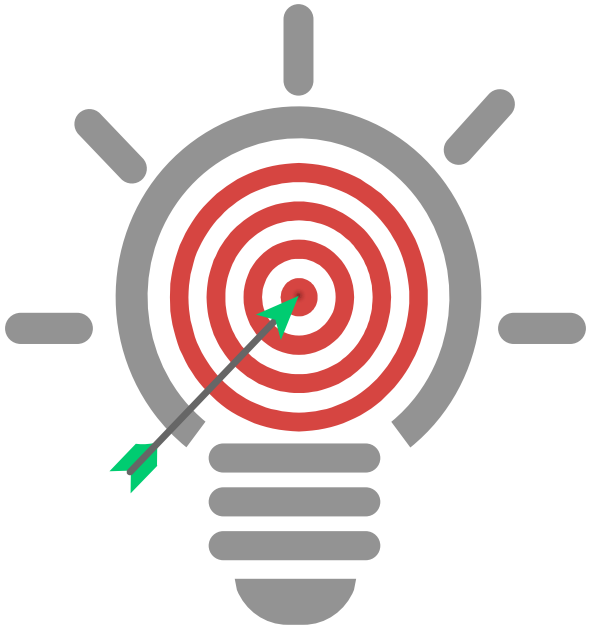
02 实验任务

03 实验步骤

04 迭代目标

实验目的

- 熟悉Java图形界面程序设计的基本方法，掌握Java Swing中的容器、常用组件和布局管理器的使用；
- 理解Java事件处理机制，掌握界面事件响应及前后端逻辑交互的基本方法；
- 理解Java多线程的概念和生命周期，掌握多线程的实现方法。

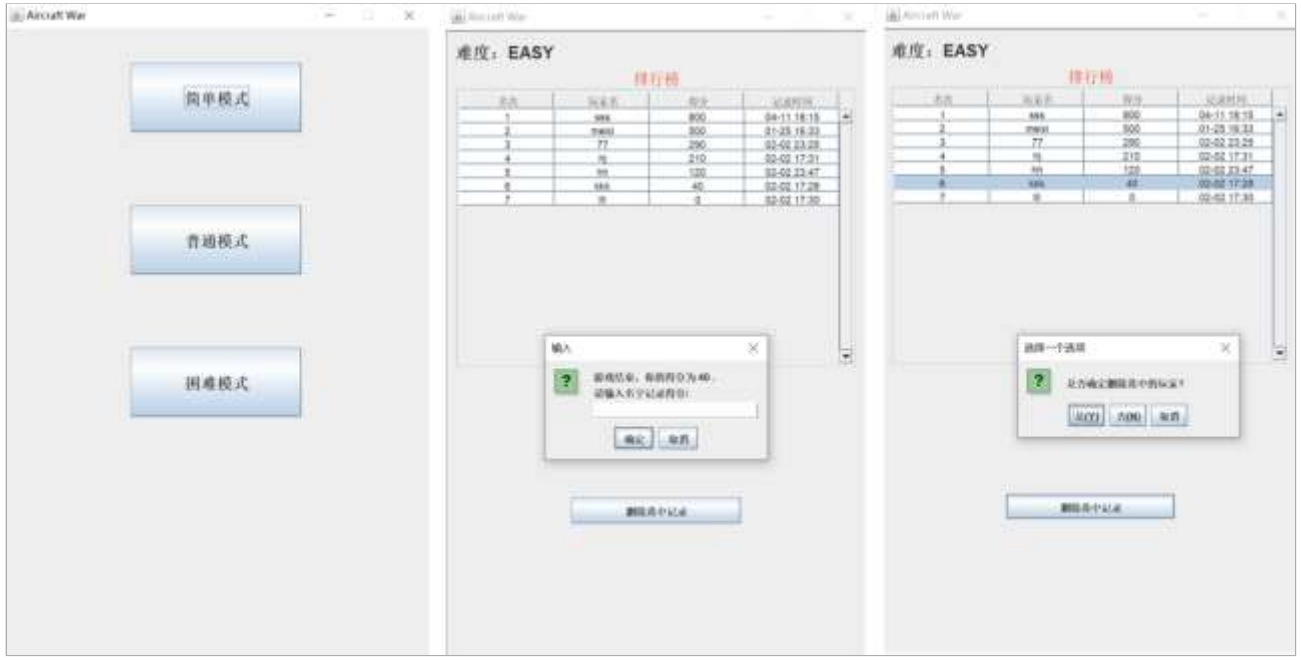


实验任务

1. 使用Java Swing类库完成以下界面：

   ✓ 游戏难度选择界面

   ✓ 得分排行榜界面



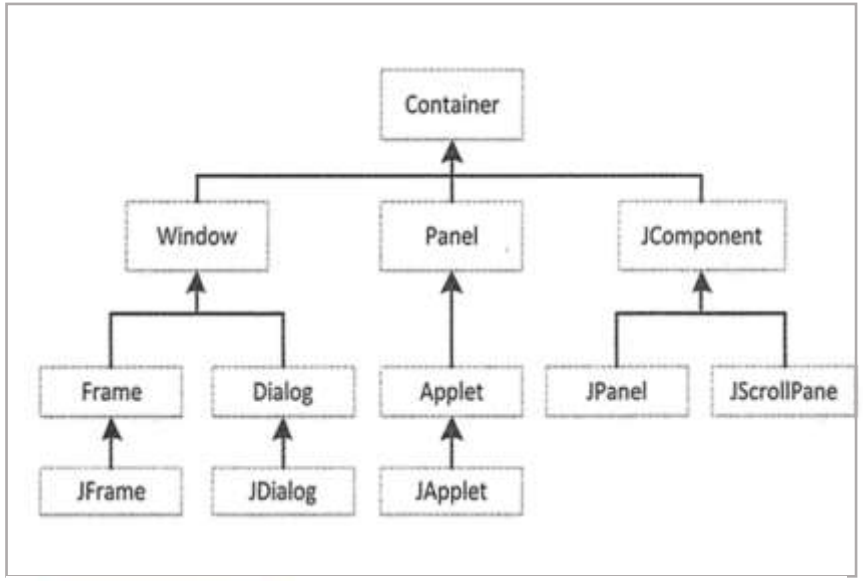
2. 使用多线程完成以下功能：

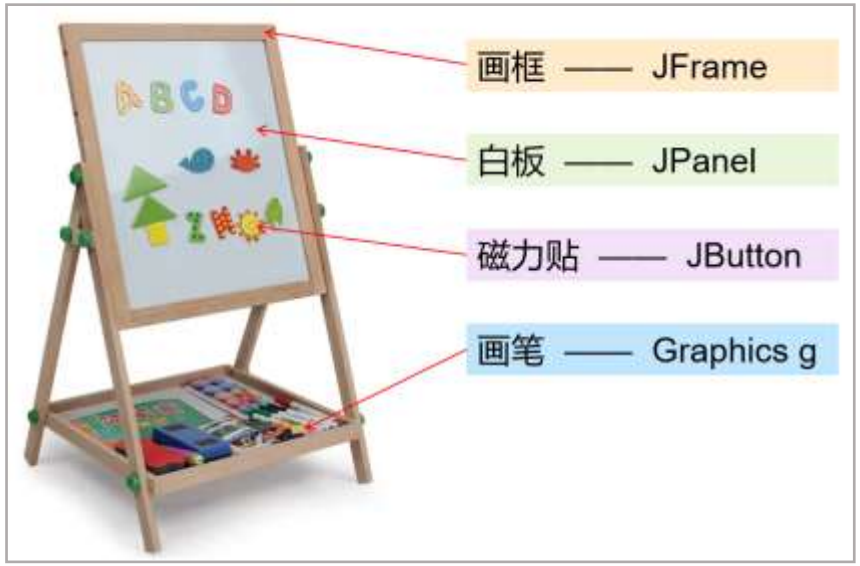
   ✓ 使用Runnable接口实现多线程，完善两种火力道具

   ✓ 继承Thread类实现多线程，完成游戏的音效控制

实验步骤：Java Swing

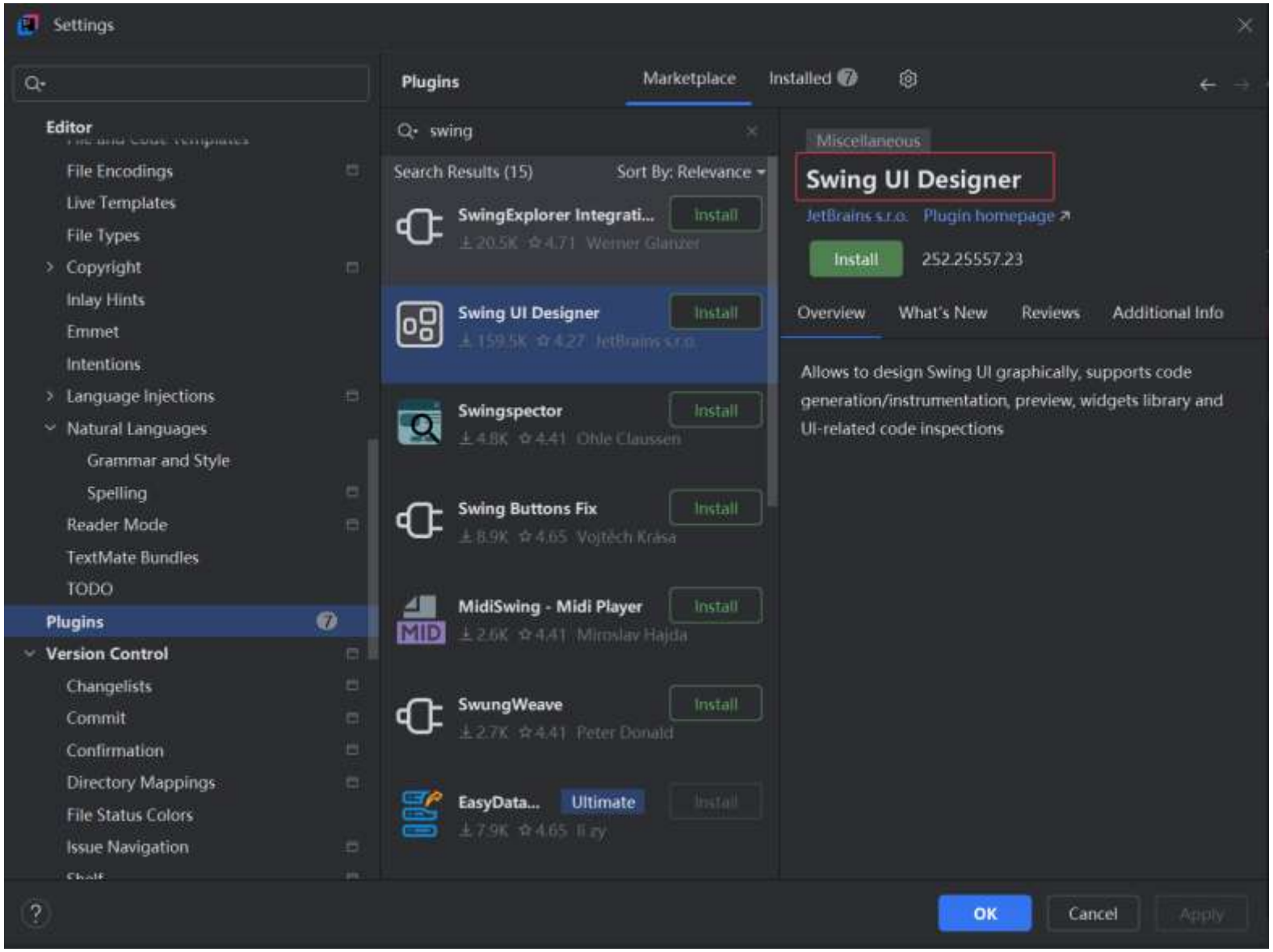
- 是 Java 为图形界面（GUI）应用开发提供的一组工具包；
- 包含构建图形界面的各种组件，可划分为容器组件和非容器组件；
- 采用MVC模式设计，实现 GUI 组件的显示逻辑和数据逻辑的分离；





实验步骤：Java Swing - Swing UI Designer（1）

- IntelliJ IDEA 提供的 Swing UI Designer可方便进行图形界面编程。



实验步骤：Java Swing - Swing UI Designer（2）

①新建Project，选中src目录，右键选择Swing UI Form，新建一个GUI Form，命名为：SimpleCalculator。

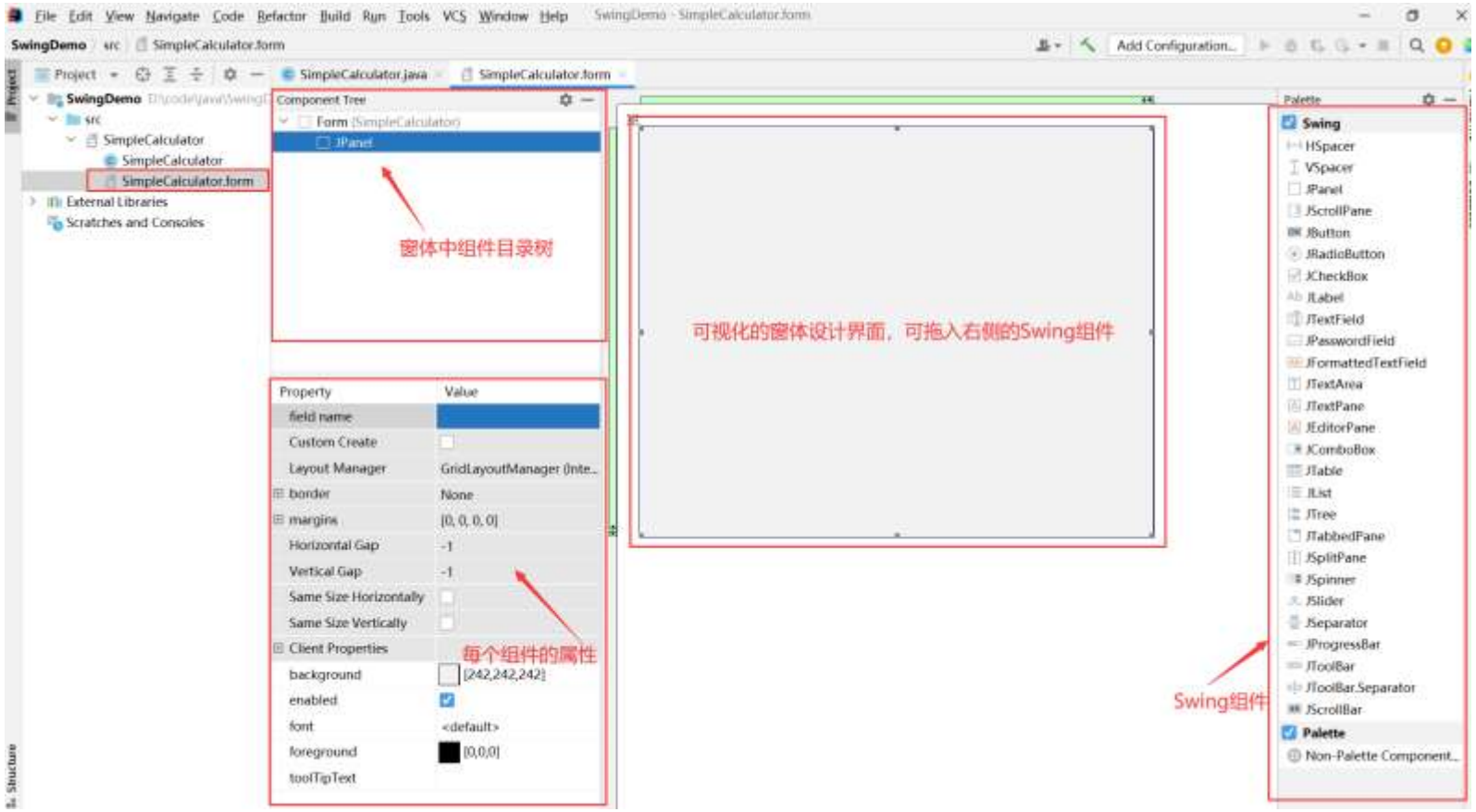
②自动生成两个文件：

- SimpleCalculator.form

用于图形界面设计；

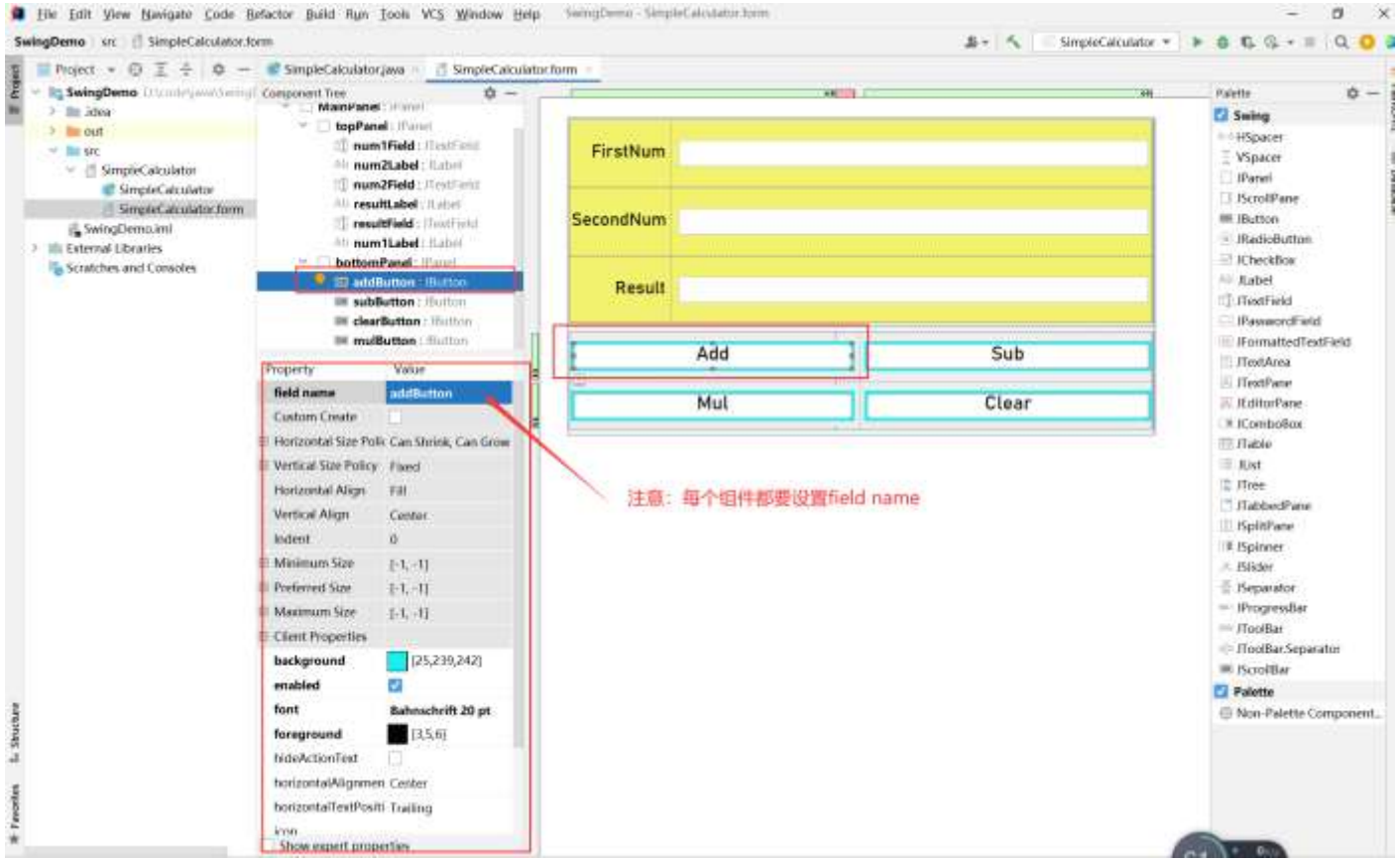
- SimpleCalculator.java

用于操作组件对象和运行。



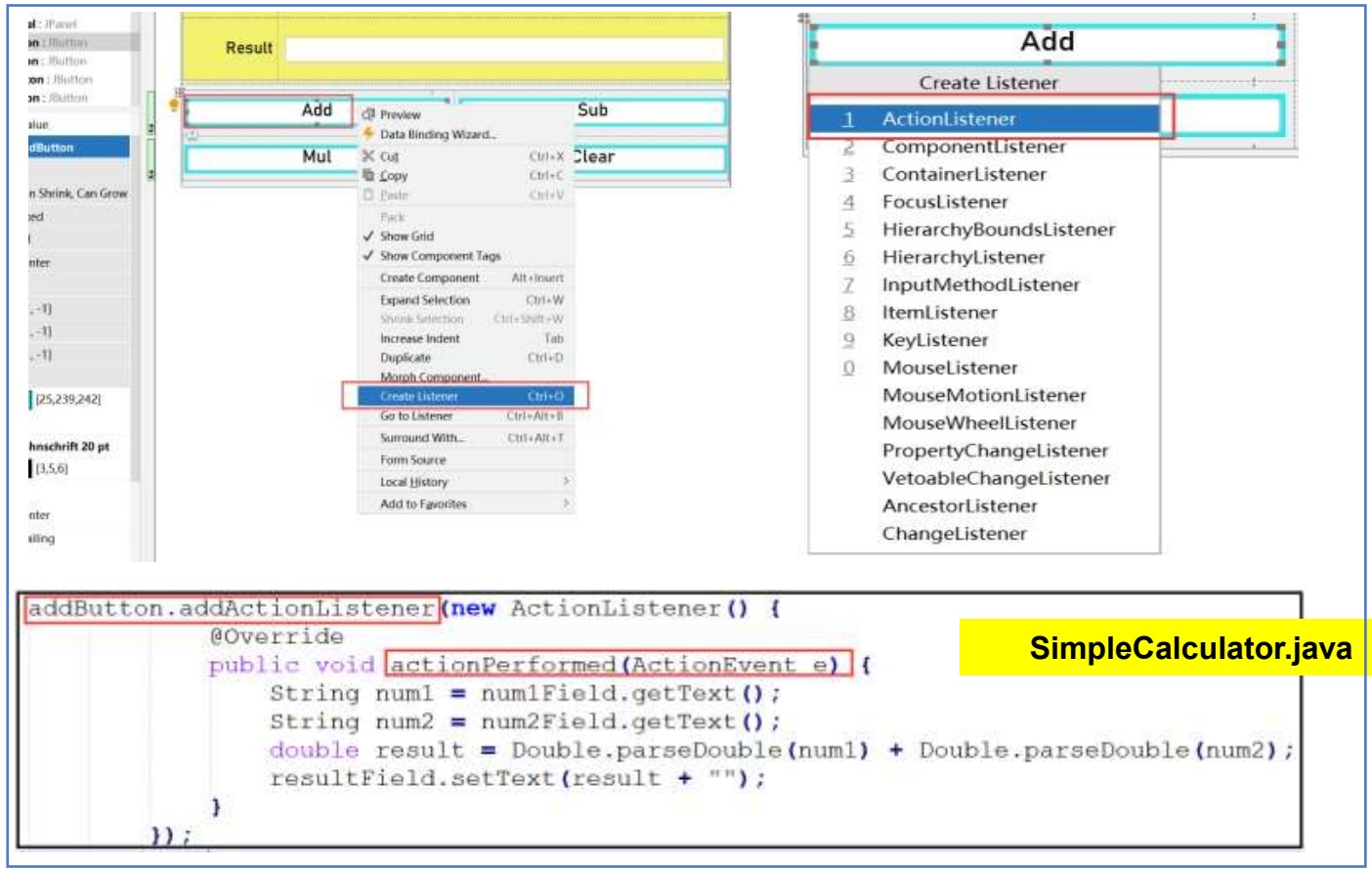
实验步骤：Java Swing - Swing UI Designer（3）

③ 在窗口设计界面中拖入所需要的Swing组件并合理布局，注意每个组件都要设置field name。右键选择Preview选项可查看窗体设计的效果。



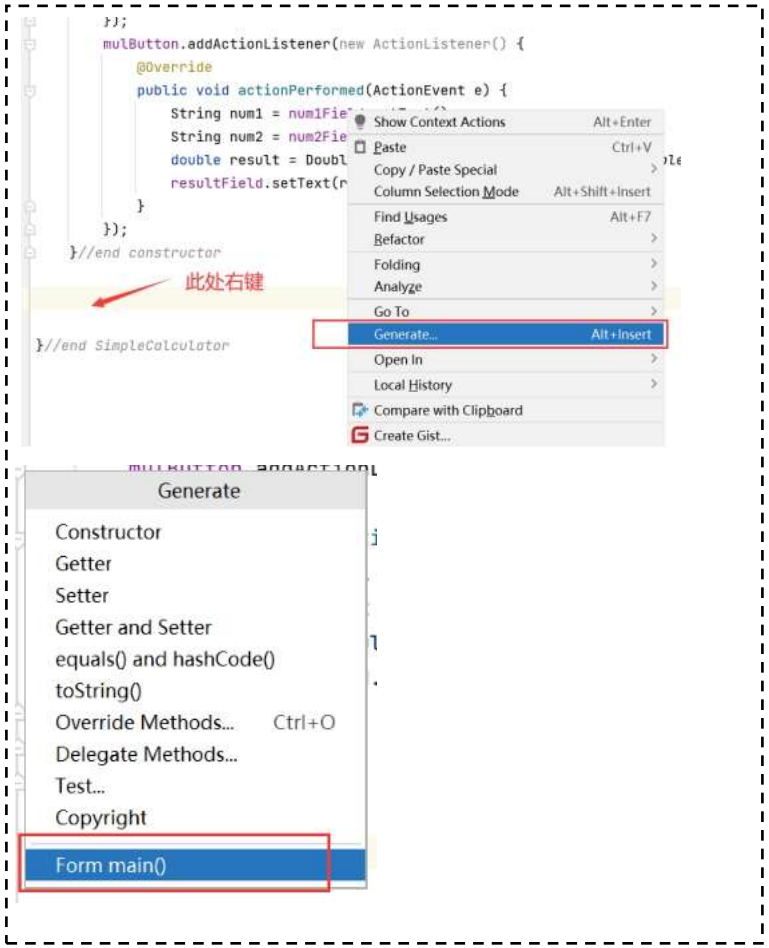
实验步骤：Java Swing - Swing UI Designer（4）

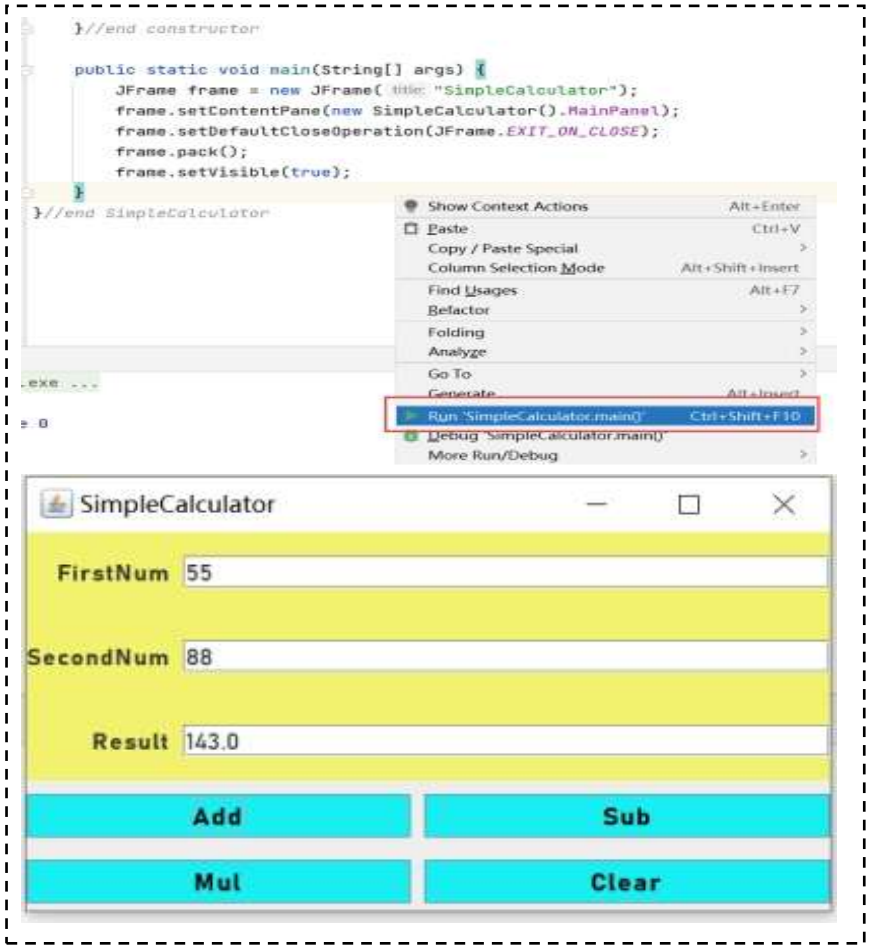
④ 添加按钮事件，右键Add按钮，选择Create Listener，为其添加ActionListener；在actionPerformed()函数体中添加事件处理代码。



实验步骤：Java Swing - Swing UI Designer（5）

⑤ 添加Form Main函数，并运行程序。

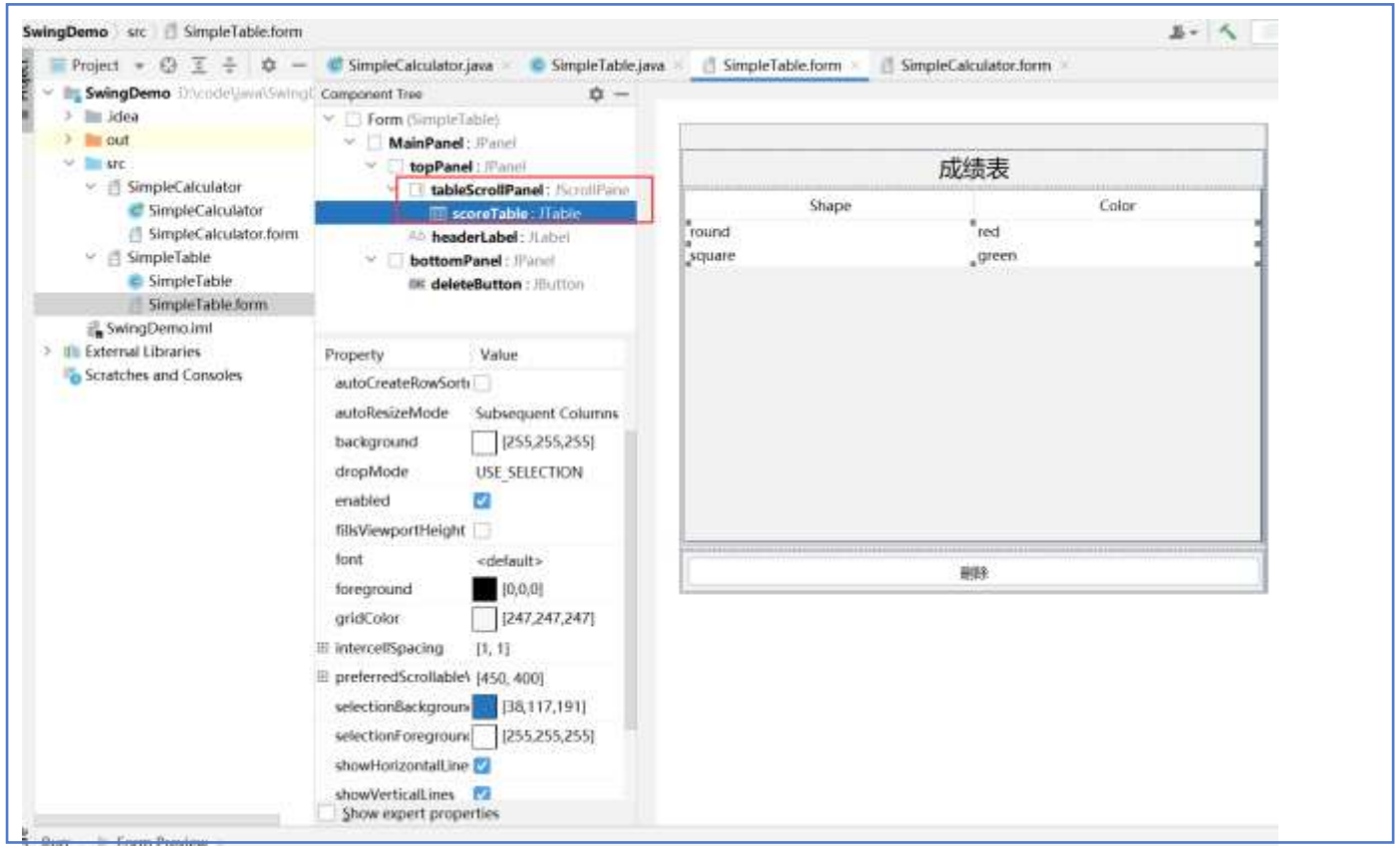




实验步骤：Java Swing - JTable（1）

➢ **JTable** 是将数据以表格的形式显示给用户看的一种组件，它包括行和列

① 添加所需组件 JScrollPane 和 JTable并合理布局。



实验步骤：Java Swing - JTable（2）

② 在构造函数中创建DefaultTableModel对象，装载数据。

```
public SimpleTable() {

        String[] columnName = {"学号","姓名","成绩"};
        String[][]tableData={{"001","Lily","78"},{"002","Jane","89"},{"003","Alex","67"},
                {"004","Macy","83"},{"005","Nancy","66"},{"006","John","99"}};

        //表格模型
        DefaultTableModel model = new DefaultTableModel(tableData, columnName){
            @Override
            public boolean isCellEditable(int row, int col){
                return false;
            }
        };

        //从表格模型那里获取数据
        scoreTable.setModel(model);
        tableScrollPanel.setViewportView(scoreTable);

    }
```

JTable表格与数据通过TableModel分离，JTable并不存储自己的数据，而是从表格模型那里获取它的数据。

实验步骤：Java Swing - JTable（3）

③ 添加“删除”按钮事件和Form Main函数。

```java
deleteButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        int row = scoreTable.getSelectedRow();
        System.out.println(row);
        int result = JOptionPane.showConfirmDialog(deleteButton,
                message: "是否确定中删除？");
        if (JOptionPane.YES_OPTION == result && row != -1) {
            model.removeRow(row);
        }
    }
});
```

实验步骤：Java Swing - CardLayout（1）

- **CardLayout** 布局管理器以时间而非空间来管理它里面的组件，能够让多个组件共享同一个显示空间

①初始化卡片布局对象cardLayout以及使用它的容器cardPanel。

```java
public class CardLayoutDemo {

    static final CardLayout cardLayout = new CardLayout( hgap: 0, vgap: 0);
    static final JPanel cardPanel = new JPanel(cardLayout);

    public static void main(String[] args) {

        JFrame frame = new JFrame( title: "CardLyaout Demo");
        frame.setSize( width: 800,  height: 1024);
        frame.setResizable(false);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        frame.add(cardPanel);

        StartMenu start = new StartMenu();
        cardPanel.add(start.getMainPanel());
        frame.setVisible(true);
    }
}
```

实验步骤：Java Swing - CardLayout（2）

② 新建窗体JFrame，并将容器cardPanel加入其中。新建StartMenu页面，加入容器cardPanel并显示。

```java
public class CardLayoutDemo {

    static final CardLayout cardLayout = new CardLayout( hgap: 0, vgap: 0);
    static final JPanel cardPanel = new JPanel(cardLayout);

    public static void main(String[] args) {

        JFrame frame = new JFrame( title: "CardLyaout Demo");
        frame.setSize( width: 800,  height: 1024);
        frame.setResizable(false);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        frame.add(cardPanel);

        StartMenu start = new StartMenu();
        cardPanel.add(start.getMainPanel());
        frame.setVisible(true);
    }
}
```

实验步骤：Java Swing - CardLayout（3）

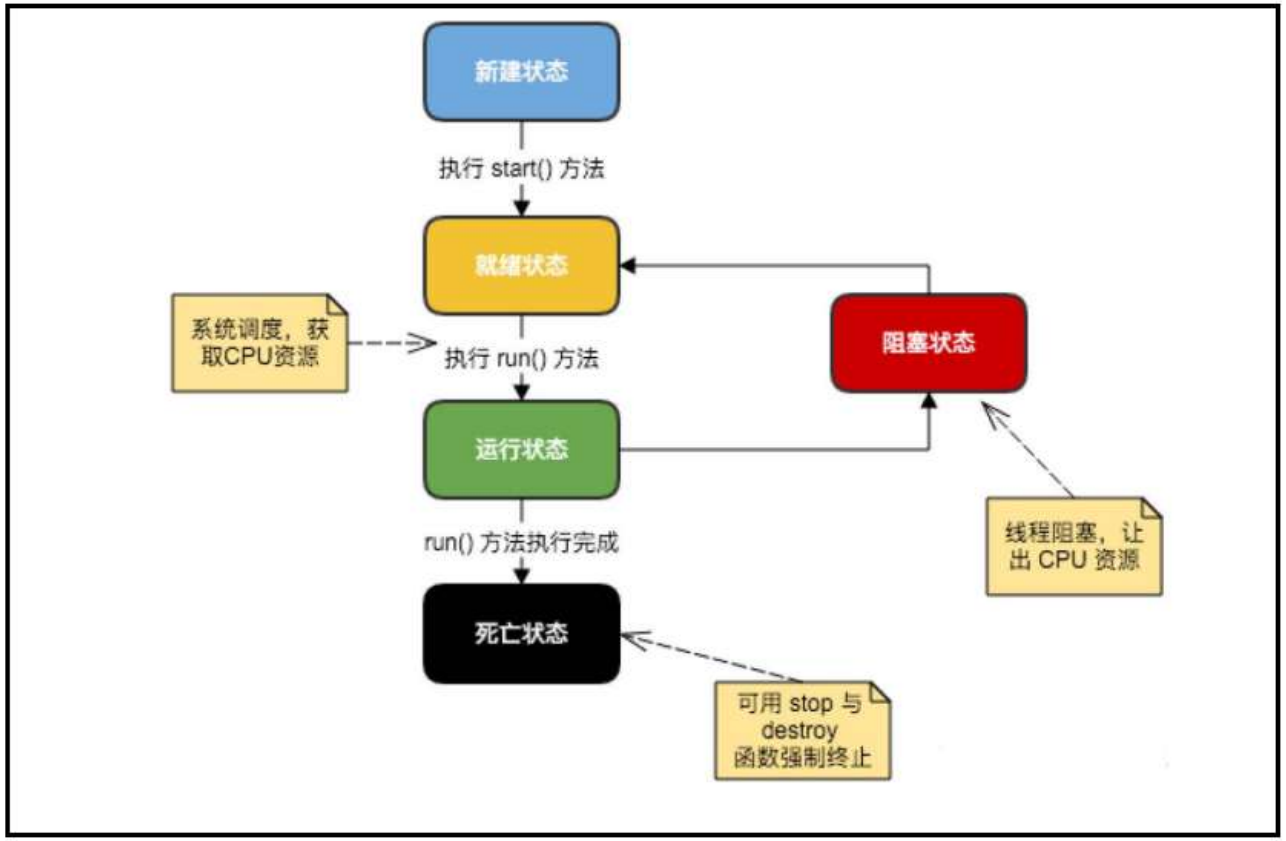
③ 为StartMenu的两个按钮添加事件，将要跳转的页面加入容器cardPanel，并使用cardLayout实现页面切换。

```java
public StartMenu() {
    simpleCalculatorButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            CardLayoutDemo.cardPanel.add(new SimpleCalculator().getMainPanel());
            CardLayoutDemo.cardLayout.last(CardLayoutDemo.cardPanel);
        }
    });
    simpleTableButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            CardLayoutDemo.cardPanel.add(new SimpleTable().getMainPanel());
            CardLayoutDemo.cardLayout.last(CardLayoutDemo.cardPanel);
        }
    });
}
```

实验步骤：Java 多线程编程（1）

- 飞机大战游戏中，火力道具生效、音效控制等功能需要用多线程来完成
- Java 给多线程编程提供了内置的支持

- 线程是一个动态执行的过程，它也有一个从产生到死亡的过程。



实验步骤：Java 多线程编程（2）

➢ Java实现多线程的两种方式：

（1）使用 Runnable 接口实现多线程，由于Runnable 是函数式接口，可使用lambda表达式简写。

```java
public class RunnableTest {
    public static void main(String[] args) {

        Runnable r = () -> {
            try {
                for (int i = 0; i < 3; i++) {
                    System.out.println("【" + Thread.currentThread().getName() +
                                       "】线程执行，当前的循环次数：" + i);

                    Thread.sleep(2000);
                }
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        };
        // 启动线程
        new Thread(r, "线程1").start
        new Thread(r, "线程2").start
    }
}
```

```
【线程 1】线程执行，当前的循环次数：0
【线程 2】线程执行，当前的循环次数：0
【线程 1】线程执行，当前的循环次数：1
【线程 2】线程执行，当前的循环次数：1
【线程 1】线程执行，当前的循环次数：2
【线程 2】线程执行，当前的循环次数：2
```

实验步骤：Java 多线程编程（3）

（2）继承 Thread 类实现多线程，继承类必须重写 run() 方法，该方法是新线程的入口点。

- 本实验已提供MusicThread类，该类继承Thread类，重写了run()方法，用于启动音频播放。

```
@Override
public void run(){
    InputStream stream =new ByteArrayInputStream(samples);
    play(stream);
}
```

- 使用方法：

```
public class ThreadTest {

    public static void main(String[] args) {
        new MusicThread("src/bgm.wav").start();
    }
}
```

注意： MusicThread 仅支持单次播放，不支持循环，且启动后无法停止。本次实验需完善音频模块，实现循环播放与停止播放功能，建议设计独立的音频管理类进行统一控制。

本次迭代开发目标（1）

任务清单
(CheckList)

难度选择界面

① 实现难度选择功能，玩家选择对应难度后，界面切换并显示该难度对应的游戏地图

注：无需细化三种难度的具体差异，仅需根据选择的难度替换对应游戏地图即可

音效控制

② 游戏运行期间循环播放背景音乐，游戏结束后停止播放；实现多种音效触发机制，包括子弹击中敌机、炸弹爆炸、道具生效、游戏结束时播放对应音效；Boss 敌机出场时循环播放其专属背景音乐，当 Boss 坠毁或游戏结束时，停止该背景音乐

本次迭代开发目标（2）

道具生效

③ 完善两种火力道具功能，道具生效时，英雄机切换弹道并持续指定时长后，自动恢复为直射状态

排行榜页面

④ 游戏结束后自动弹出得分排行榜界面，同步弹出输入窗口供玩家填写姓名，支持玩家记录本局得分，或删除任意一条历史得分记录

同学们，请开始实验吧！

THANK YOU